Department of Computer and Software Engineering
SE: Machine Learning

Course Instructor:	Date:
Day:	Semester:

Lab 11: Refactoring, MLflow and Deployment

Lab Title	CLO	BT	PLO	Weightage
Refactoring & Deployment MLflow, APIs, Serialization				

Name	Roll Number	Report /10	Viva /5	Total /15
Zunaira Abdul Aziz	BSSE23058			

Checked on: _____

Signature: _____

Objectives

- Refactor unstructured ML code into modular functions
- Track experiments using MLflow
- Serialize trained models
- Build REST APIs using FastAPI

Problem Statement

In real-world machine learning systems, models are not built as isolated scripts but as part of structured, maintainable, and deployable pipelines.

You are given a poorly written ("spaghetti") machine learning script that mixes data loading, preprocessing, model training, and evaluation in a single block of code. Such code is difficult to maintain, extend, debug, and deploy.

Your task is to transform this code into a clean and modular pipeline by separating concerns into well-defined functions.

After refactoring, you will extend the pipeline to include:

- **Experiment Tracking:** Use MLflow to log model parameters, performance metrics, and artifacts.
- **Model Serialization:** Save the trained model to disk and reload it for inference.
- **Model Deployment:** Build a REST API using FastAPI to serve predictions from the trained model.

You will use the dataset from the previous lab (Lab 10) to ensure continuity.

By the end of this lab, you will have a complete mini machine learning system that includes:

- Clean, modular code
- Experiment tracking
- Persistent models
- A working prediction API

Part A: Refactoring

Task 1: Identify Issues

Open train bad.py and list at least four problems related to structure and main-tainability (e.g. hard-coded paths, no functions, magic numbers, mixed concerns).

Mixed Concerns (Spaghetti Code): The script lacks a clear separation of tasks, combining data loading, preprocessing, model training, and evaluation in a single, unstructured block of code.

No Modular Functions: There are no defined functions, making it impossible to reuse specific parts of the pipeline (like preprocessing or evaluation) in other scripts or for real-time predictions.

Magic Numbers and Hard-coded Parameters: The script uses "magic numbers" for model hyperparameters such as `max_depth=100`, `min_samples_split=2`, and `min_samples_leaf=1` directly in the code rather than using a configuration file or variables.

Dead and Inefficient Code: The script includes "pointless multiplication" on numerical columns and contains unused data structures (like `unused_list` and `unused_dict`) and redundant print statements that clutter the code without adding value [`train_bad.py`].

Hard-coded File Paths: The data source is hard-coded as `"lab10_data.csv"`, which makes the script fragile if the file is moved or renamed.

Task 2: Modularization

Refactor the script into the following functions inside lab11_starter.py:

- load_data() — read lab10_data.csv and return a DataFrame.
- preprocess_data(df) — drop the categorical city column, then split into feature matrix X (all columns except target) and label vector y.
- train_model(X, y) — fit and return a DecisionTreeClassifier.
- evaluate_model(model, X, y) — compute and return the accuracy score (expected ≥ 0.8).
- run_pipeline() — orchestrate the above steps and save the model to disk.

Part B: MLflow

Task 3: Logging
Inside run_pipeline(), wrap the training and evaluation steps in an MLflow run and log the following:

- **Parameters:** model hyper-parameters (e.g. max_depth, criterion).
- **Metrics:** training accuracy (key: accuracy; must reach the expected_accuracy threshold of 0.8).
- **Model:** the fitted DecisionTreeClassifier via mlflow.sklearn.log_model.

Part C: Serialization

Task 4: Save and Load Model
Use joblib to persist the trained model:

- In run_pipeline(): call joblib.dump(model, "model.joblib") after training.
- In the FastAPI section: call joblib.load("model.joblib") to restore the model before the app starts serving requests.

```python
def load_data():
    """
    Load the dataset from the CSV file.

    Returns:
        pd.DataFrame: The loaded dataframe containing the data.
    """
    df = pd.read_csv("lab10_data.csv")
    return df


def preprocess_data(df):
    """
    Preprocess the input dataframe by cleaning and preparing features and
target.

    Args:
        df (pd.DataFrame): The raw dataframe loaded from the CSV.

    Returns:
        tuple: A tuple containing (X, y) where X is the feature matrix and y is
the target vector.
    """
    if "city" in df.columns:
        df = df.drop(columns=["city"])

    X = df.drop("target", axis=1)
    y = df["target"]
    return X, y


def train_model(X, y):
    """
    Train a machine learning model using the provided features and target.

    Args:
        X (pd.DataFrame or np.ndarray): The feature matrix.
        y (pd.Series or np.ndarray): The target vector.

    Returns:
        object: The trained model object.
    """
    model = DecisionTreeClassifier(
        random_state=42, max_depth=100, min_samples_split=2, min_samples_leaf=1
    )
    model.fit(X, y)
    return model


def evaluate_model(model, X, y):
    """
```

```
    Evaluate the trained model on the given data and return the accuracy.

    Args:
        model (object): The trained model.
        X (pd.DataFrame or np.ndarray): The feature matrix for evaluation.
        y (pd.Series or np.ndarray): The true target values.

    Returns:
        float: The accuracy score of the model.
    """
    predictions = model.predict(X)
    acc = accuracy_score(y, predictions)
    return acc


def run_pipeline():
    """
    Run the complete ML pipeline: load data, preprocess, train, evaluate, and
save the model.

    Returns:
        float: The accuracy of the trained model on the training data.
    """
    df = load_data()
    X, y = preprocess_data(df)

    with mlflow.start_run():
        model = train_model(X, y)
        acc = evaluate_model(model, X, y)

        mlflow.log_param("max_depth", 100)
        mlflow.log_param("random_state", 42)

        mlflow.log_metric("accuracy", acc)

        mlflow.sklearn.log_model(model, "decision_tree_model")

        joblib.dump(model, "model.joblib")

        print(f"Pipeline complete. Accuracy: {acc}")
        return acc
```

Part D: FastAPI

Task 5: API Endpoint

Complete the /predict POST endpoint in lab11_starter.py:

- **Request body:** {"features": [age, income, hour, leak_feature]}
- **Response:** {"prediction": 0} or {"prediction": 1}
- Raise HTTPException(status_code=422) if the features key is missing.
- Raise HTTPException(status code=503) if the model has not been loaded.

Verify the API is reachable via the GET / endpoint which returns {"message": "ML Model API is running"}.

```
app = FastAPI()

try:
    model = joblib.load("model.joblib")
except:
    model = None


@app.get("/")
def home():
    # Returns a simple status message to confirm the API is reachable.
    return {"message": "ML Model API is running"}


@app.post("/predict")
def predict(data: dict):
    """
    Run inference for a single sample.

    Expected JSON body:
        { "features": [age, income, hour, leak_feature] }

    Returns:
        dict: { "prediction": 0 or 1 }
    """
    if "features" not in data:
        raise HTTPException(
            status_code=422, detail="Missing 'features' key in request body"
        )

    if model is None:
        raise HTTPException(
            status_code=503, detail="Model not loaded. Please run the pipeline
first."
        )
```

```
    try:
        features = np.array(data["features"]).reshape(1, -1)
        prediction = model.predict(features)
        return {"prediction": int(prediction[0])}
    except Exception as e:
        raise HTTPException(status_code=400, detail=f"Inference error:
{str(e)}")
```

Discussion Questions

- Why is modular design important?

  It breaks spaghetti code into independent pieces, making the system easier to debug, test, and update without breaking the entire project.

- What are benefits of MLflow?

  It provides an automated memory for experiments, allowing you to track which hyperparameters led to the best accuracy and saving the resulting model artifacts for deployment.

- Why is model serialization required?

  It allows you to freeze a trained model's state (weights and parameters) to a file so it can be moved from a training environment to a production server without needing to retrain it.